

Linked List

Reason:

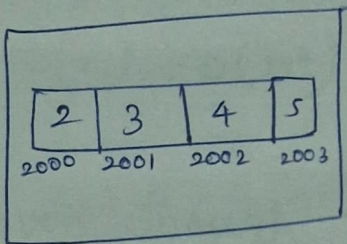
- ① In Array, its length is fixed
- ② Cannot increase or decrease the size of the array

But for accessing elements by its index is faster than LL.

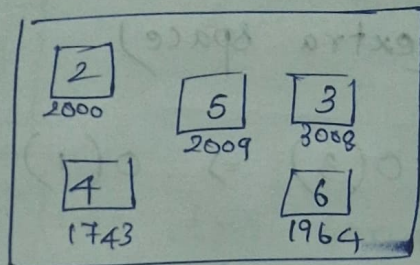
Linked List

* It is similar data structure as array, but it is not contiguous

Array



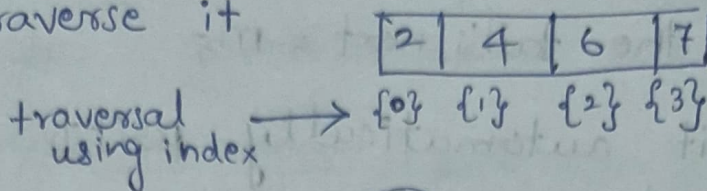
Linked List



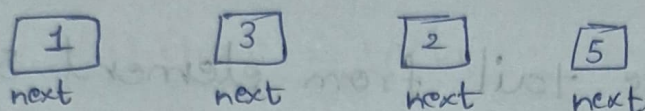
* Their size can be increased/ at any time decreased

How do we store it

① In array we store element, by using index we can traverse it



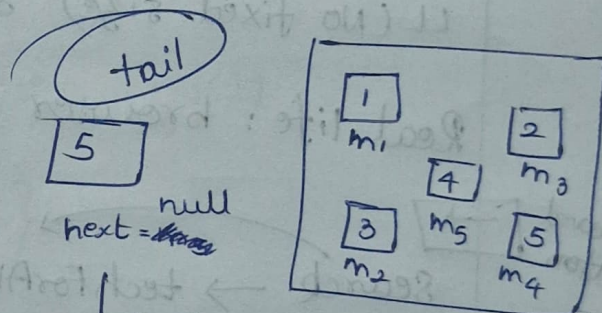
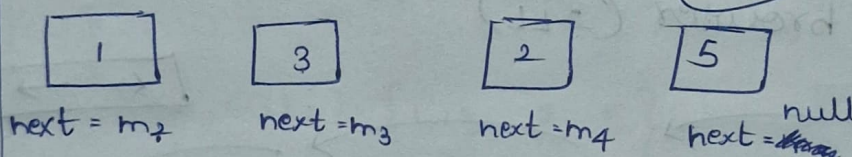
② But in LL, we store one element and 2nd without we store one more thing called next



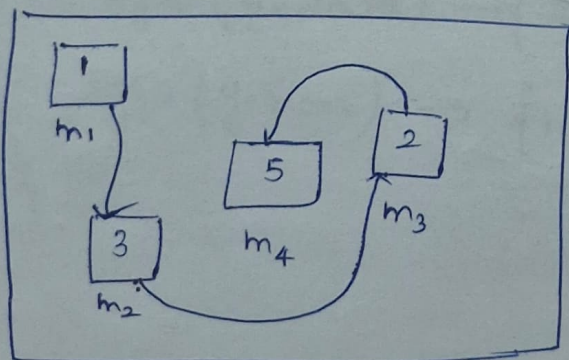
→ it is the address of next element } through this traversal can be done

* Starting pt of the LL is called Head of LL

Now head = m_1



Invisibly creating a link



→ since there is nothing beyond 5, do the next of 5 will be

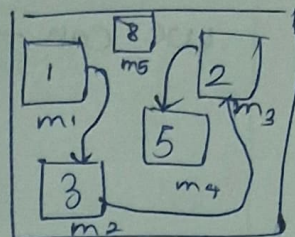
NULL

Now we need to add element 8 to LL

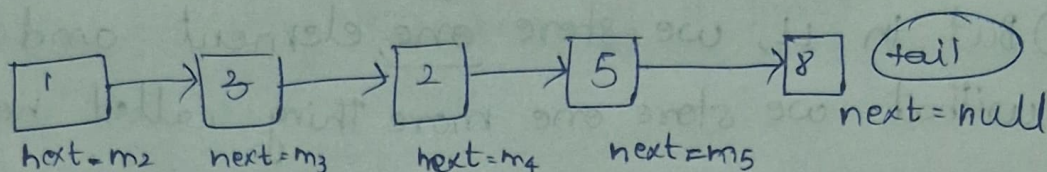
Now, we know that tail's next is null

change the tail's next = m_5

Then it automatically points element 8



head = m_1



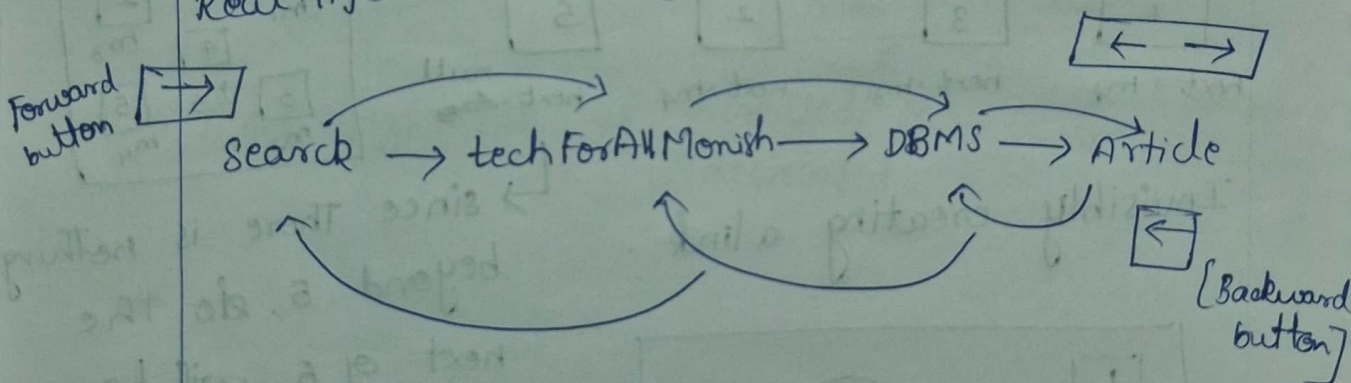
Then change the tail from element 5 to element 8, and next = null [There is no element beyond]

Where it used

DS: Stack, Queue

LL (No fixed size) same for stack, Queue

Real life: browser (also LL)



self defined Object

class Node

```
{ int data;  
  Node next;  
}
```

Node(int data1, Node next1)

```
{ this.data = data1;  
  this.next = next1;  
}
```

```
}
```

Node(int data1)

```
{ this.data = data1;  
  this.next = null;  
}
```

```
}
```

public class LinkedList

```
{ public static void main(String args[])  
  {
```

```
    int arr[] = {1, 3, 2, 5}
```

```
    Node y = new Node(arr[2])
```

```
    syso(y.data) -> { 2 } //
```

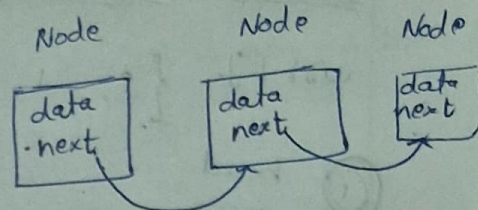
```
    syso(y.next) -> { null } //
```

```
  }
```

```
}
```

Node
data(int)
next(add)

but we don't the data type of this, so we go for self-defined Obj



memory space

32-bit

int $\rightarrow$ 4 bytes

pt $\rightarrow$ 4 "

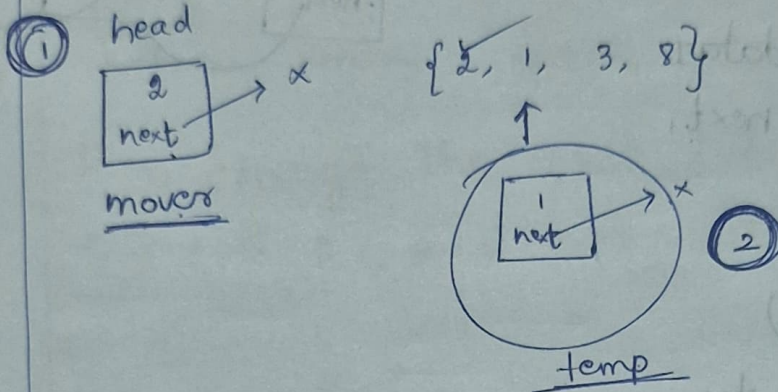
64-bit

int $\rightarrow$ 4 bytes

pt $\rightarrow$ 8 bytes

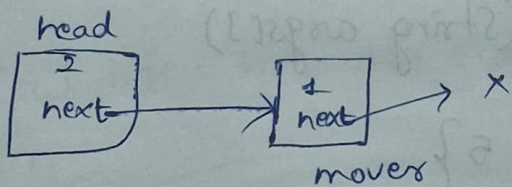
Convert array to LL

arr[] = {2, 1, 3, 8}



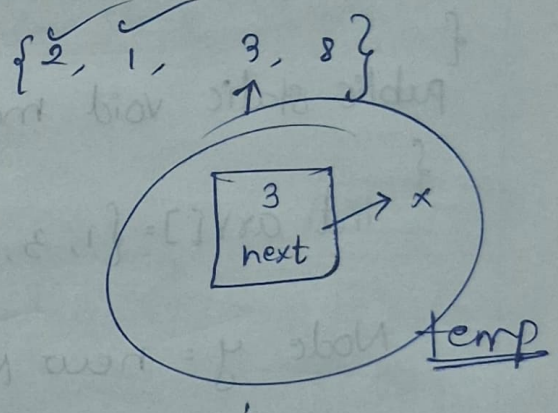
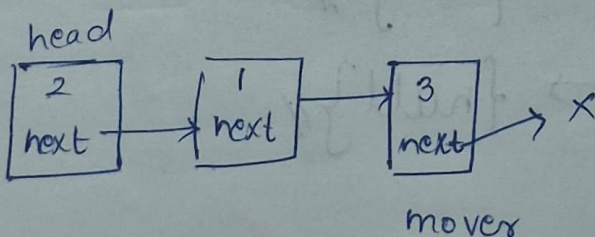
In order to create LL, the previous instead of pointing to null, it should point 1

✓ mover.next = temp ③

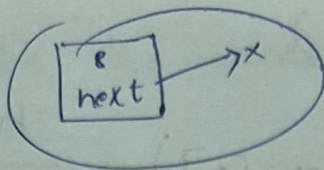


Then mover, ④

✓ mover.next = temp

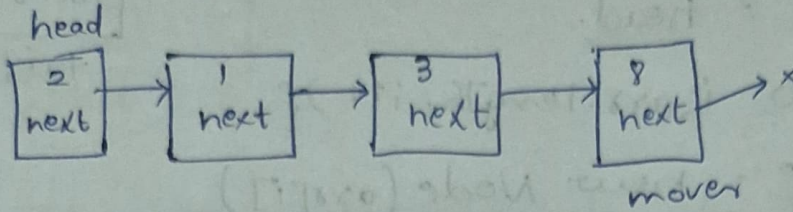


{2, 1, 3, 8}



temp

main.next = temp



Coding : Convert Array to LL and return head

class Node {

int data;
Node next;

Node (int data)

```

{
    this.data = data;
    this.next = NULL;
}
  
```

Node (int data, Node next)

```

{
    this.data = data;
    this.next = next;
}
  
```

class Main

```

{
    public static void main (String args[])
    {
        int arr[] = {1, 2, 3, 4, 5}

        Node head = ConvertToLL (arr)

        System.out.println (head)
    }
}
  
```



```
private static Node (int arr)
```

```
{
```

```
① Node head = new Node(arr[0]);
```

```
Node tail = head;
```

```
for(int i=0; i<arr.length; i++){
```

```
② Node temp = new Node(arr[i])
```

```
③ tail.next = temp
```

```
④ tail = temp
```

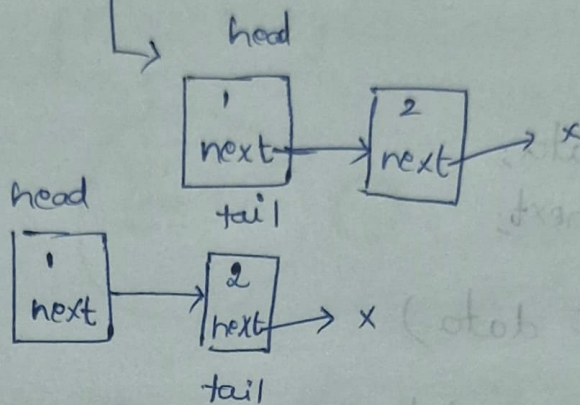
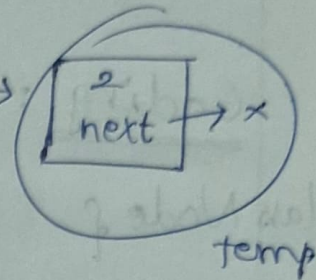
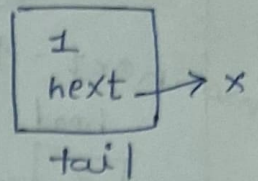
```
}
```

```
return head;
```

```
}
```

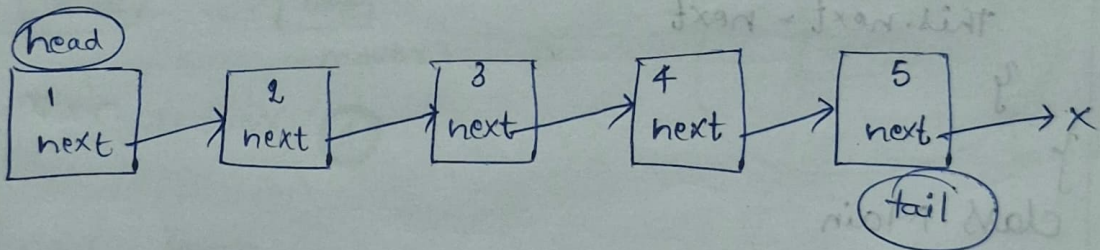
```
}
```

head



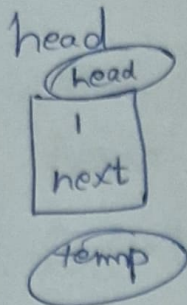
Traversing Through LL

Now LL, look like This



head → Starting point of LL

① take the variable temp which also points

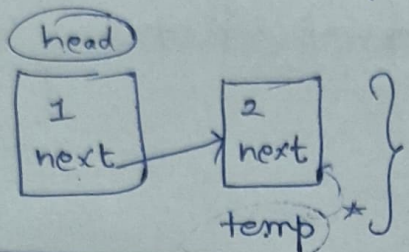


} then print temp.data = ①

②

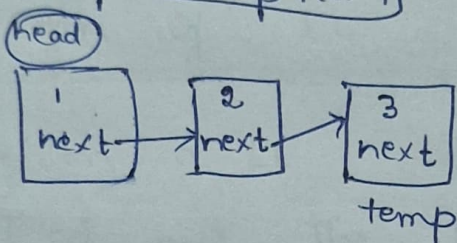
Now to print next element
(which next element add)

Temp = temp.next



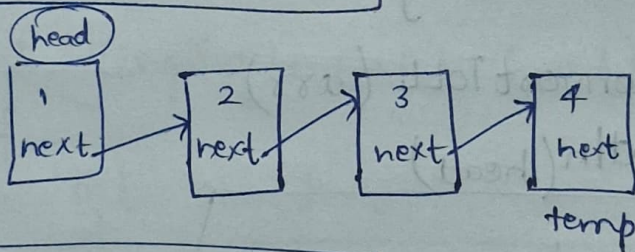
temp.data = ②

* temp = temp.next



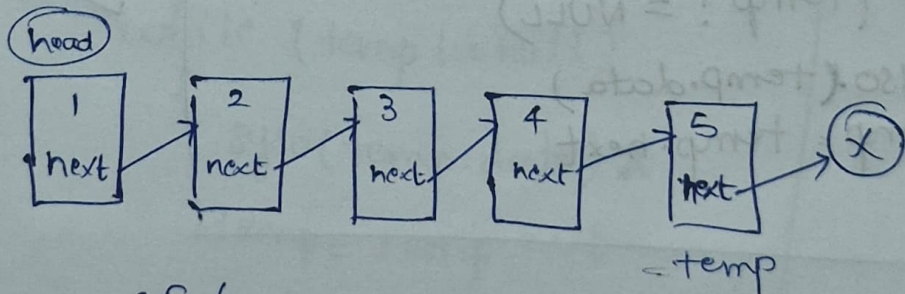
temp.data = ③

temp = temp.next



temp.data = ④

temp = temp.next



temp.data = ⑤

if (temp.next) == null

STOP

class Node {

int data;

Node next;

Node(int data, Node next)

{ this.data = data;

this.next = next;

}

Node(int data)

{ this.data = data;

this.next = null;

}

}

class main {

psvm(String args[])

{ int arr[] = {1, 2, 3, 4, 5}

Node head = ConvertToLL(arr)

System.out.println(head);

Node temp = head;

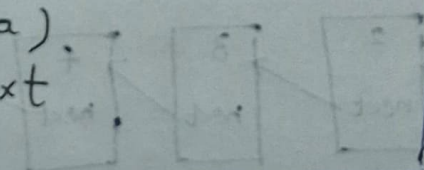
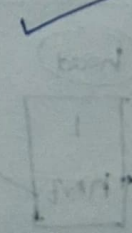
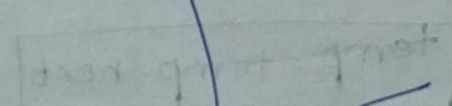
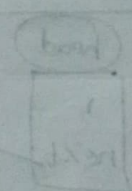
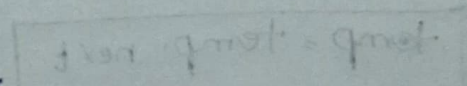
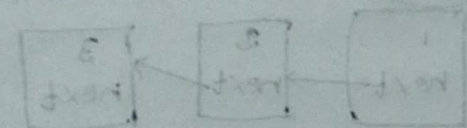
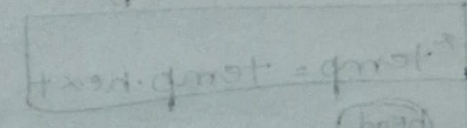
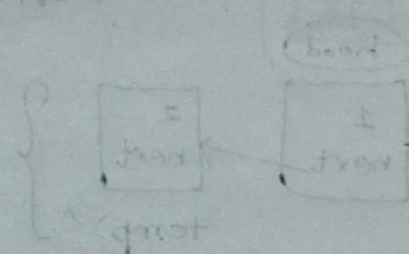
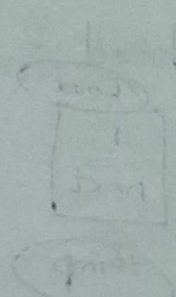
while (temp != NULL)

{ sysout(temp.data);

temp = temp.next;

}

}




```
private static Node ConvertToLL(int arr)
```

```
{ Node head = new Node(arr[0])
```

```
Node tail = head
```

```
for(int i=0; i<arr.length; i++)
```

```
{ Node temp = new Node(arr[i])
```

```
tail.next = temp
```

```
tail = temp
```

```
}
```

```
}
```

Length of LL

→ given a LL

→ Find out the Length, where head is given

```
ps int lengthOfLL(Node head)
```

```
{
```

```
int count = 0;
```

```
Node temp = head
```

```
while (temp != null)
```

```
{
```

```
syso(temp.data) ✗
```

```
temp = temp.next
```

```
count++
```

```
}
```

```
return count;
```


Whenever it is given a LL

↳ Always head is given

Search in LL

→ Find The value is present or Not

private static CheckIfPresent (Node head, int val)

```
{
    Node temp = head
    while (temp != null)
    {
        if (temp.data == val) return 1
        temp = temp.next
    }
```

return 0;

}

class main

$O(N)$
↓
worst Case
(Not found)

$O(1)$
↓
element
in head
(Best Case)

$O(N/2)$
↓
average